

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3067 – Redes de Computadoras

Sección 10

Ing. Kevin Velásquez



Laboratorio 2

Detección y Corrección de Errores en L2 OSI

Angie Nadissa Vela López, 23764

Paula Daniela de León Godoy, 23202

GUATEMALA, 02 de agosto de 2026

1. Descripción de la práctica

Este laboratorio implementa un sistema emisor-receptor que simula la capa de Enlace de datos frente a un canal no confiable. El objetivo es transmitir un mensaje de texto y garantizar (o al menos verificar) su integridad usando dos algoritmos con propósitos distintos:

- Código de Hamming SEC (Single Error Correction): agrega bits de paridad que permiten corregir automáticamente 1 bit de error.
- CRC-32 (IEEE 802.3, polinomio 0x04C11DB7): agrega un residuo de 32 bits que permite detectar errores, pero no corregirlos.

El sistema sigue una arquitectura de 5 capas, implementadas en dos lenguajes distintos que se comunican por sockets TCP reales:

Capa	Servicios implementados	Dónde
Aplicación	solicitar_mensaje, mostrar_mensaje	Emisor y receptor
Presentación	codificar_mensaje (texto → ASCII binario) / decodificar_mensaje	Emisor / receptor
Enlace	calcular_integridad / verificar_integridad + corregir_mensaje	Emisor / receptor
Ruido	aplicar_ruido: flip de cada bit con probabilidad configurable (ej. 1/100)	Emisor
Transmisión	enviar_informacion / recibir_informacion (sockets TCP)	Emisor / receptor

El emisor está escrito en Python y actúa como cliente de socket; el receptor está escrito en C++ y actúa como servidor, escuchando en un puerto hasta que el emisor se conecta y envía la trama ya afectada por el ruido simulado. El CRC-32 implementado usa división polinomial directa con el polinomio estándar IEEE 802.3, sin las reflexiones de bits ni el XOR inicial/final de 0xFFFFFFFF que usa la variante "Ethernet/zlib"; es una versión simplificada pero matemáticamente equivalente en su lógica de detección.

2. Resultados

Se realizaron dos tipos de prueba:

- Corridas reales del emisor y receptor conectados por socket.
- Una simulación de 300 corridas por punto usando la misma lógica de codificación/decodificación, para poder graficar el comportamiento con una muestra estadísticamente más amplia de la que permite hacer todo a mano.

2.1 Corridas reales

Caso 1 (Sin ruido, Hamming, mensaje "Hola")

```
receptor.cpp
38 // Decodifica un codeword de Hamming; si detecta un error de 1 bit lo corrige.
47 HammingResult hammingDecode(const string& codeword) {
48     int n = (int)codeword.size();
49     vector<int> code(n + 1, 0);

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$ g++ -std=c++17 -O2 -o receptor receptor.cpp
nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$ ./receptor

RECEPTOR
Puerto para escuchar [5000]:
Escuchando en el puerto 5000... (esperando al emisor)
Trama recibida (38 bits): 11001001100001110111101101100
010100001

--- Resultado en el receptor ---
Algoritmo: Hamming SEC
Sin errores.
Mensaje recibido: Hola

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$ python3 emisor.py
EMISOR
Mensaje a enviar (texto): Hola
Algoritmo de integridad:
1) Hamming SEC (correccion)
2) CRC-32 (deteccion)
Opcion (1/2): 1
Tasa de error del canal (ej. 1/100, o 0 para sin ruido): 0
Host del receptor [127.0.0.1]:
Puerto del receptor [5000]:

--- Resumen de envio ---
Texto original      : Hola
Bits (ASCII)       : 01001000011011110110110001100001
Algoritmo          : Hamming SEC
Trama (sin ruido)   : 11001001100001110111101101100010100001
(38 bits)
Trama (con ruido)   : 11001001100001110111101101100010100001
(0 bit(s) volteado(s))

Trama enviada a 127.0.0.1:5000.
nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$
```

Caso 2 (Ruido bajo (1/50), Hamming, mensaje "Hi": se corrige exitosamente 1 bit alterado)

```
receptor.cpp
38 // Decodifica un codeword de Hamming; si detecta un error de 1 bit lo corrige.
47 HammingResult hammingDecode(const string& codeword) {
48     int n = (int)codeword.size();
49     vector<int> code(n + 1, 0);

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$ ./receptor

RECEPTOR
Puerto para escuchar [5000]:
Escuchando en el puerto 5000... (esperando al emisor)

Trama recibida (21 bits): 000010011000011001001

--- Resultado en el receptor ---
Algoritmo: Hamming SEC
Se detecto y corrigio un error en la posicion 2 antes de
decodificar.
Mensaje recibido: Hi

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$
nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$ python3 emisor.py
EMISOR
Mensaje a enviar (texto): Hi
Algoritmo de integridad:
1) Hamming SEC (correccion)
2) CRC-32 (deteccion)
Opcion (1/2): 1
Tasa de error del canal (ej. 1/100, o 0 para sin ruido): 1/50
Host del receptor [127.0.0.1]:
Puerto del receptor [5000]:

--- Resumen de envio ---
Texto original      : Hi
Bits (ASCII)       : 0100100001101001
Algoritmo          : Hamming SEC
Trama (sin ruido)   : 010010011000011001001 (21 bits)
Trama (con ruido)   : 000010011000011001001 (1 bit(s) voltead
o(s))

Trama enviada a 127.0.0.1:5000.
nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$
```

Caso 3 (Ruido alto (1/10), CRC-32, mensaje "Test": se detecta el error y se descarta la trama)

```
receptor.cpp
38 // Decodifica un codeword de Hamming; si detecta un error de 1 bit lo corrige.
47 HammingResult hammingDecode(const string& codeword) {
48     int n = (int)codeword.size();
49     vector<int> code(n + 1, 0);
...

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$ ./receptor
RECEPTOR
Puerto para escuchar [5000]:
Escuchando en el puerto 5000... (esperando al emisor)
Trama recibida (64 bits): 010100001110010111110011011101
001110000100001001110110111111001

--- Resultado en el receptor ---
Algoritmo: CRC-32
Error detectado. No fue posible entregar el mensaje.
Trama descartada.
nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$

nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$ python3 emisor.py
EMISOR
Mensaje a enviar (texto): Test
Algoritmo de integridad:
1) Hamming SEC (correccion)
2) CRC-32 (deteccion)
Opcion (1/2): 2
Tasa de error del canal (ej. 1/100, o 0 para sin ruido): 1/10
Host del receptor [127.0.0.1]:
Puerto del receptor [5000]:

--- Resumen de envio ---
Texto original      : Test
Bits (ASCII)       : 01010100011001010111001101110100
Algoritmo          : CRC-32
Trama (sin ruido)   : 0101010001100101011100110111010001
10011001101010111011001 (64 bits)
Trama (con ruido)   : 0101000011100101111100110111010001
00001001110110111111001 (9 bit(s) volteado(s))
-----

Trama enviada a 127.0.0.1:5000.
nadissa@Nadissa08:~/tareas/redes/Lab2_Redex$
```

2.2 Overhead (bits de redundancia)

Hamming necesita muchos menos bits de redundancia que CRC-32, y su overhead crece de forma logarítmica con el tamaño del mensaje (*r tal que $2^r \geq m + r + 1$*). CRC-32 siempre agrega exactamente 32 bits, sin importar el tamaño del mensaje, por lo que su overhead relativo baja rápidamente en mensajes largos.

Mensaje	Bits de datos	Trama Hamming (bits)	Redundancia Hamming	Trama CRC-32 (bits)	Redundancia CRC-32
Corto (2 car.)	16	21	5 (31%)	48	32 (200%)
Medio (11 car.)	88	95	7 (8%)	120	32 (36%)
Largo (40 car.)	312	321	9 (3%)	344	32 (10%)

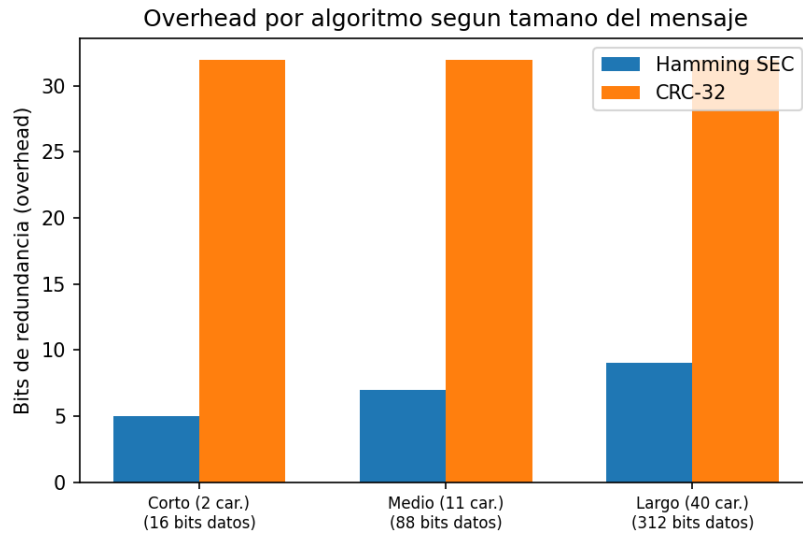


Figura 1. Bits de redundancia por algoritmo según el tamaño del mensaje.

2.3 Tasa de éxito y falla silenciosa vs. probabilidad de error

Para cada combinación de algoritmo y tamaño de mensaje se corrieron 300 transmisiones simuladas por cada tasa de error, contando qué porcentaje se entregó correctamente.

Tasa de entrega correcta (%):

Tasa de error	Corto (2 car.)	Medio (11 car.)	Largo (40 car.)
0 (sin ruido)	100%	100%	100%
1/200	100%	94%	53%
1/100	98%	78%	18%
1/50	91%	41%	1%
1/20	72%	5%	0%
1/10	40%	0%	0%
1/7	17%	0%	0%
1/5	5%	0%	0%

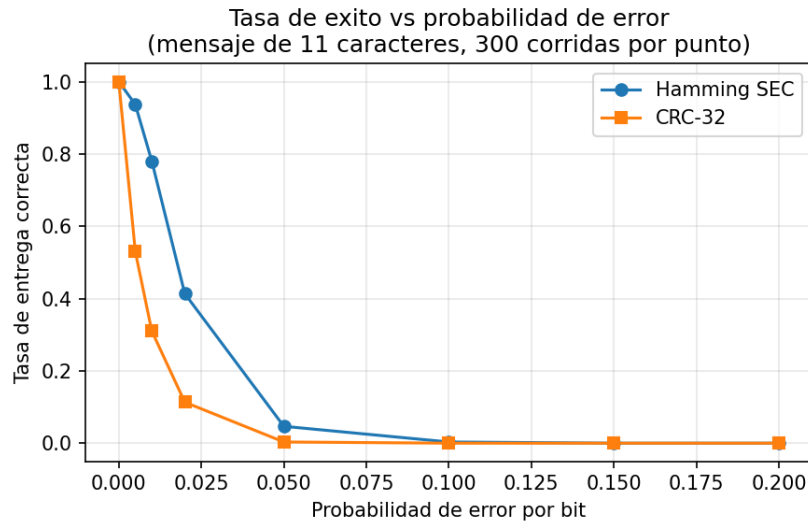


Figura 2. Tasa de éxito vs. probabilidad de error (mensaje de 11 caracteres).

Más importante que la tasa de éxito es la tasa de falla silenciosa, es decir, casos donde el receptor entrega un mensaje incorrecto sin avisar que hubo un error. Aquí es donde se ve la diferencia estructural entre ambos algoritmos:

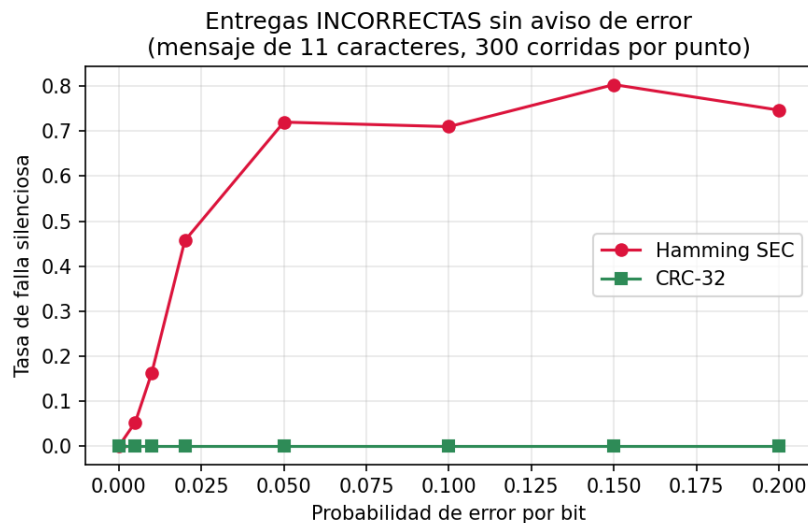


Figura 3. Entregas incorrectas sin aviso de error (mensaje de 11 caracteres).

CRC-32 se mantuvo en 0% de fallas silenciosas en las 3 categorías de tamaño y las 8 tasas de error probadas (2,400 corridas por algoritmo). Hamming SEC, en cambio, empieza a fallar silenciosamente en cuanto la probabilidad de más de 1 error por trama deja de ser despreciable, y esa tasa de falla silenciosa crece con el tamaño del mensaje porque tramas más largas tienen más oportunidades de sufrir 2 o más errores.

3. Discusión

¿Es posible engañar al algoritmo?

Sí, a ambos, pero con probabilidades muy distintas:

- Hamming SEC solo garantiza corrección de 1 bit de error. Si ocurren 2 errores en la misma trama, el síndrome calculado casi siempre apunta a una tercera posición distinta a las dos que realmente fallaron: el receptor corrige un bit que estaba bien, deja los 2 bits erróneos sin tocar, y entrega datos incorrectos reportando "sin errores" o "error corregido" con total confianza. Se comprobó experimentalmente: con solo 2 bits alterados en una trama corta, el receptor corrigió la posición equivocada y entregó datos distintos a los originales sin ninguna advertencia.
- CRC-32 solo se engaña si el patrón exacto de bits alterados coincide con un múltiplo del polinomio generador de 32 bits. Es matemáticamente posible, pero la probabilidad es del orden de $1/2^{32}$ para un patrón de error aleatorio, en la práctica, nunca ocurrió en ninguna de las corridas simuladas.

Ventajas y desventajas

Criterio	Hamming SEC	CRC-32
Capacidad	Corrige 1 bit de error automáticamente, sin retransmisión	Solo detecta; requiere descartar/retransmitir
Redundancia	Baja y crece logarítmicamente (5-9 bits en los mensajes probados)	Fija en 32 bits, independiente del tamaño del mensaje
Confiabilidad con 2+ errores	Débil: puede fallar silenciosamente (ver Figura 3)	Muy robusta: detecta prácticamente cualquier patrón de error
Complejidad de implementación	Moderada: hay que calcular posiciones de paridad y decodificar el síndrome	Moderada: requiere implementar división polinomial módulo 2
Costo computacional	$O(n \cdot r)$, r pequeño (crece con $\log n$)	$O(n \cdot 33)$, constante por bit procesado

Con los datos obtenidos, Hamming SEC tuvo mejor desempeño solo en canales con muy baja probabilidad de error, donde su capacidad de autocorrección evita una retransmisión. Para cualquier canal más ruidoso, CRC-32 fue claramente más confiable, nunca entregó datos corruptos sin avisar, mientras que Hamming lo hizo hasta en un 80% de los casos con canales muy ruidosos y mensajes largos.

CRC-32 también es más flexible ante mayores tasas de error, precisamente porque no intenta adivinar una corrección. Cuando no puede garantizar integridad, simplemente descarta la trama en vez de arriesgarse a entregar datos incorrectos.

Gracias a los resultados obtenidos, deducimos que es mejor utilizar la corrección de errores (Hamming) en canales donde retransmitir es costoso o lento y la tasa de error esperada es baja. La detección de errores (CRC-32) conviene cuando la retransmisión es barata y rápida (como en una red local), y es la opción preferida cuando la integridad de los datos es más importante que la latencia. Tras una breve investigación, descubrimos que por eso Ethernet y la mayoría de los protocolos de red usan CRC en lugar de códigos correctores.

Comentario grupal

Este laboratorio fue interesante porque permitió ver en la práctica las diferencias estructurales entre corrección y detección de errores, mientras que Hamming SEC es elegante en overhead, CRC-32 es mucho más confiable en canales ruidosos. Implementar ambos algoritmos en lenguajes distintos comunicándose por sockets reforzó la importancia de una arquitectura de capas bien definida. Cabe mencionar que la guía resumida (basada en copiar/pegar manual entre emisor y receptor) habría sido significativamente más fácil de implementar que la versión completa con sockets y ruido probabilístico, aunque esta última ofrece una simulación mucho más realista del problema.

4. Conclusiones

- Los algoritmos de corrección y detección de errores resuelven problemas distintos, uno prioriza disponibilidad y el otro confiabilidad.
- Hamming SEC es eficiente en overhead pero frágil ante ráfagas de 2 o más errores, y su falla característica es silenciosa (el peor tipo de falla en un sistema real, porque no hay forma de saber que ocurrió sin un mecanismo de verificación).
- CRC-32 tiene mayor costo fijo en bits de redundancia, pero su probabilidad de fallo silencioso es despreciable, lo que lo hace la opción más segura como mecanismo único de integridad.
- La arquitectura de capas permitió probar cada algoritmo de forma aislada y reutilizar la misma capa de transmisión por sockets para ambos.

5. Repositorio

Todo el código desarrollado se encuentra en el siguiente [repositorio de Github](#).

6. Referencias

- Hamming, R. W. (1950). Error detecting and error correcting codes. The Bell System Technical Journal, 29(2), 147–160.
- IEEE Computer Society. (2018). IEEE Standard for Ethernet (IEEE Std 802.3-2018). Sección de Frame Check Sequence (CRC-32, polinomio 0x04C11DB7).
- Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson. Capítulo 6: The Link Layer and LANs.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson. Capítulo 3: The Data Link Layer.